

Getting external access to BuddyPress's notifications (for Knowledge Commons)

Cite as: Martin Paul Eve, "Getting external access to BuddyPress's notifications (for Knowledge Commons)", *eve.gd: Martin Paul Eve*, February 18, 2025, <https://doi.org/10.59348/gpbaz-kx09>.



As part of my work on [Knowledge Commons](#), I want to make more of our development process open, welcoming, and transparent, by using blogging. So I will be writing some technical posts on what I'm doing there and how I've overcome various technical challenges. In this post, I want to set out how I got BuddyPress notifications into a separate application (our new "Profiles" app).

First: why am I doing this? We currently have a monolithic WordPress stack and it's pretty unwieldy. It isn't easy to make swift changes if we want to add a new profile field or display things differently. We really need to modularise the system so that it's manageable, scalable, and secure. This means moving away from the massive WordPress-does-everything model. We already have this in part with our new repository system, [KCWorks](#), which runs on Invenio.

However, we are NOT going to be able to move everything at once. We need to lift and shift components one-by-one and leave the core of the application still functioning. This means that while we will move some data OUT of WordPress, other portions will need to remain inside WordPress (and managed by it and its database) until they are too are ported. This poses significant difficulties for external applications. For instance, in the case that this blog post is going to address, we have "notifications" of "activity" inside WordPress. But when a user views someone else's profile

page, we want a dropdown telling the logged-in user if they have any notifications. To do this, we have to wire our new application up to the WordPress database and figure out how to read BuddyPress's notifications.

How to do this?

Well, Django, the framework we are using for the new Profiles application, supports multiple named databases. So, for example, we have these:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql_psycopg2",
        "NAME": "django",
        "USER": "django",
        "PASSWORD": "django",
        "HOST": "postgres.database.host",
        "PORT": "5432",
        "OPTIONS": {
            "connect_timeout": 5,
        },
    },
    "wordpress": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": "mysql",
        "USER": "mysql",
        "PASSWORD": "mysql",
        "HOST": "mysql.database.host",
        "PORT": "3306",
        "OPTIONS": {
            "connect_timeout": 5,
        },
    },
}
```

This gives us two databases. However, we need to be very careful. We do *not* want to accidentally write to the MySQL/WordPress database; we want it to be read only. We also want to make sure that only specific models are fetched and displayed from the WordPress database – and the rest come from our Postgres install (to modularise the data).

How do we do that? We write a database router.

```

class ReadWriteRouter:
    """
    A router to control all database operations on models in the WordPress
    DB
    """
    def db_for_read(self, model, **hints):
        """
        Controls which database should be used for read operations for a given
        model.
        """
        return (
            "wordpress"
            if model.__name__.lower().startswith("wp")
            else "default"
        )
    def db_for_write(self, model, **hints):
        """
        Controls which database should be used for writes for a given model.
        """
        return None if model.__name__.lower().startswith("wp") else "default"
    def allow_migrate(self, db, app_label, model_name=None, **hints):
        """
        Controls if a model should be allowed to migrate on the given database.
        """
        return db == "default"

```

Then in settings.py:

```

DATABASE_ROUTERS = ["newprofile.wordpress_router.ReadWriteRouter"]

```

Essentially, this says: any model with a name that starts with “wp” should be routed to the “wordpress” database for reads and should return NO writing capacity if it’s a request to write.

So we now have a way of connecting to the WordPress database. How, though, does BuddyPress store its notifications? They’re in a table called wp_bp_notifications. So I created a Django model that reflects this table:

```

class WpBpNotification(models.Model):
    """
    A model for a WordPress notification
    """
    id = models.BigAutoField(primary_key=True)
    user = models.ForeignKey(
        "WpUser", on_delete=models.CASCADE, db_column="user_id"
    )
    item_id = models.BigIntegerField()
    secondary_item_id = models.BigIntegerField(null=True)
    component_name = models.CharField(max_length=75)
    component_action = models.CharField(max_length=75)
    date_notified = models.DateTimeField()
    is_new = models.BooleanField(default=False)
    class Meta:
        """
        Metadata for the WpBpNotification model
        """
        db_table = "wp_bp_notifications"
        managed = False
        indexes = [
            models.Index(fields=["item_id"]),
            models.Index(fields=["secondary_item_id"]),
            models.Index(fields=["is_new"]),
            models.Index(fields=["component_name"]),
            models.Index(fields=["component_action"]),
            models.Index(fields=["user", "is_new", name="useritem"]),
        ]

```

So, pretty obviously, the way to select new notifications is to pull out items where the user is the logged-in user and the `is_new` field is `True`. In this case, [I have also linked the user id to a model that represents the WordPress user \(WpUser\)](#).

So far so good. But how does BuddyPress change what we get here (a load of `item_id`, `secondary_item_id`, `component_name`, `component_action` columns) into a nice formatted message: “You have 8 new followers”? The answer is, with some distributed trickery.

BuddyPress records the “`component_name`” and “`component_action`” in the database and then, when it comes to getting a message for this, it asks all registered components whether or not they can handle a particular notification type. This means that the code for different types of

notification is spread over many different files and they all handle messages in a distributed fashion. So, for example, in buddypress/bp-friends/bp-friends-notifications.php you have this block of code:

```
case 'friendship_request':
    $link = bp_loggedin_user_domain() . bp_get_friends_slug() .
'/requests/?new';
    $action = 'request';
    // Set up the string and the filter.
    if ( (int) $total_items > 1 ) {
        /* translators: %d: the number of pending requests */
        $text = sprintf( __( 'You have %d pending friendship requests',
'buddypress' ), (int) $total_items );
        $amount = 'multiple';
    } else {
        /* translators: %s: friend name */
        $text = sprintf( __( 'You have a friendship request from %s',
'buddypress' ), bp_core_get_user_displayname( $item_id ) );
        $amount = 'single';
    }
    break;
```

So how do we handle this in an external application? Well, the way I did it was to work out what the possible values were from the database by doing a distinct select:

Possible values:

```
comment_reply -- deprecated?: rarely used
deposit_published -- deprecated: from CORE
deposit_review -- deprecated: from CORE
friendship_accepted -- deprecated?: rarely used
friendship_request -- deprecated?: rarely used
group_invite -- unsure if used
join_mla_forum -- unsure if used
membership_request_accepted -- possibly used
membership_request_rejected -- possibly (but rarely) used
member_promoted_to_admin -- possibly used
member_promoted_to_mod -- possibly (but rarely) used
newsletter_opt_out -- possibly (but rarely) used
new_at_mention -- possibly (but rarely) used
new_follow -- used a lot
new_group_site_member -- used a lot
new_membership_request -- used a lot
new_message -- possibly (but rarely) used
new_user_email_settings -- used a lot
update_reply -- possibly (but rarely) used
```

I then did counts on these to determine whether we are actually using them and which ones seem the most important. It was then a matter of grepping through our codebase to find which of these were custom, which were built-in, and writing handlers for them.

There are also (as per that friendship request example) some special cases where instead of displaying 25 different messages (“X is following you”, “Y is following you”, “Z is following you”), we want to say “3 new users followed you”. In such cases we have to count the number of unread follows and return that message instead – and then stop processing new follow messages.

The eventual result for how I did this is in the [notifications.py](#) file in my newprofiles app. The above code snippets also come from [models.py](#) and [wordpress_router.py](#). Calls to the notification system are made from [api.py](#) and are used in [the base template](#).

Overall: modularisation is hard. At the moment, this application is pretty tightly coupled to WordPress (although there is the option in the code to disable all WordPress database access, so we can always do a quick test to judge how dependent we are on the WordPress core). Over time, though, this will enable us to rewrite parts of the code so they are not locked to one central technology and so that we can share aspects like notifications across a dispersed ecosystem.